



5



19



2280 sp

نظري

كلية الهندسة المعلوماتية

السنة الثالثة

SQL server part 1

د. روان قرعوني



RB Informatcs; 18/6/2025

محتوى مجاني غير مخصص للبيع التجاري

قواعد معطيات

بسم الله نبدأ

في المحاضرات السابقة، بدأنا بدراسة الجانب النظري من قواعد البيانات ، ومررنا بثلاث مراحل أساسية :

مخطط الكيان-العلاقة (ERD):

حيث تعرّفنا على كيفية تصميم قواعد البيانات من منظور مفاهيمي، عبر تحديد الكيانات (Entities) والعلاقات (Relationships) والسمات (Attributes)، وهي المرحلة التي تُستخدم فيها الرسوم التخطيطية لوصف بنية البيانات.

1

التحويل إلى نموذج علائقي (Relational Model):

بعد بناء ERD، قمنا بتحويله إلى جداول (Relations) تمثل كل كيان وعلاقته، مع تحديد المفاتيح الأساسية (primary key) والخارجية (foreign key)، لنصل إلى تمثيل منظم للبيانات.

2

الجبر العلائقي (Relational Algebra):

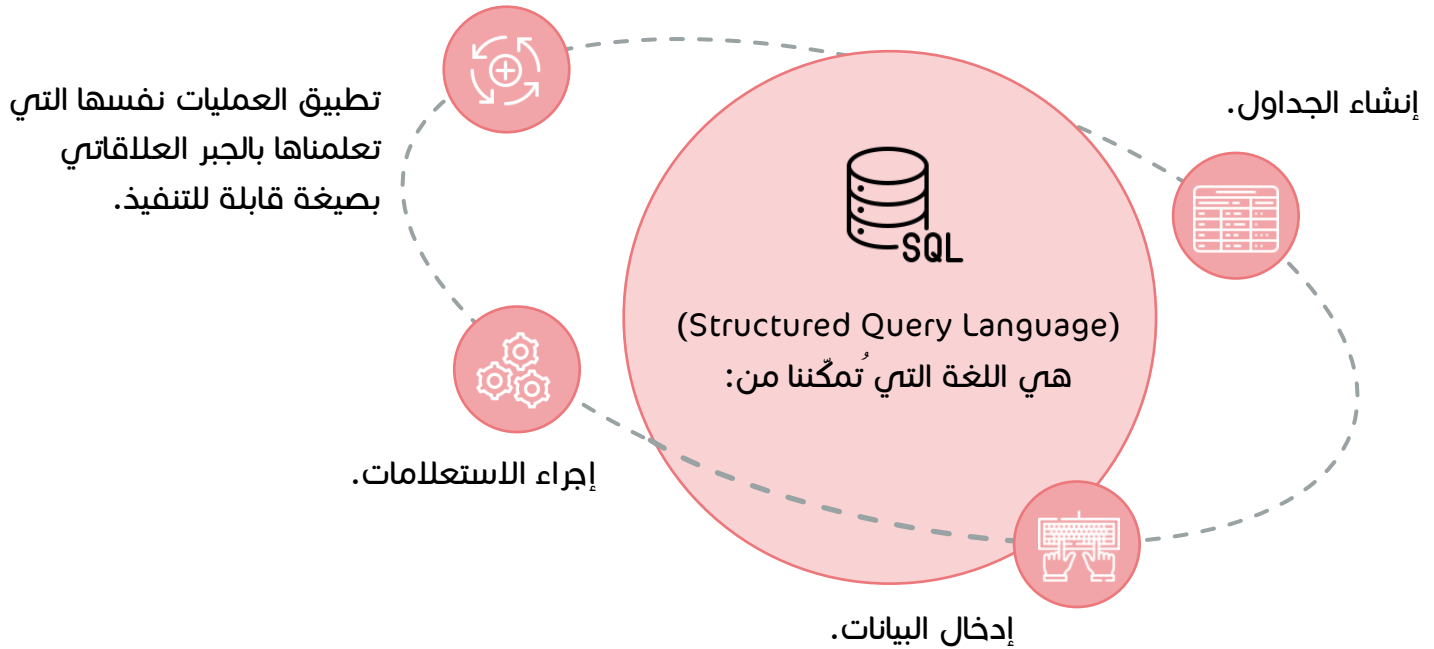
و هو الإطار الرياضي الذي استخدمناه للتعامل مع العلاقات، من خلال عمليات مثل: الإسقاط (π) والاختيار (σ) - الانضمام ($\Join$) - الفرق (-) - القسمة ($\div$) وغيرها.. وقد مكّنا هذا من فهم منطق الاستعلامات وكيفية استخراج البيانات بشكل دقيق.

3



❧ في هذه المحاضرة :

سننتقل من النماذج النظرية إلى التطبيق العملي باستخدام SQL Server :
وهو أحد أنظمة إدارة قواعد البيانات العلائقية (RDBMS) المستخدمة على نطاق واسع في المؤسسات .



❧ الأفكار الرئيسية لهذه المحاضرة :

1- أنواع البيانات (Data Types)

لفهم كيفية تخزين البيانات بطريقة صحيحة وفعّالة (نصوص، أرقام، تواريخ...).

2- إدارة هيكل الجداول

CREATE Table, DROP, ALTER

لإنشاء الجداول أو تعديلها أو حذفها - وهي عمليات مرتبطة بتحويل ال ERD إلى قاعدة بيانات عملية

3- تعديل البيانات

INSERT, UPDATE, DELETE

لإضافة أو تعديل أو حذف البيانات داخل الجداول.

4- أوامر الاستعلام الأساسية

SELECT, FROM , WHERE

لإستخراج البيانات بناءً على شروط محددة، مثل عمليات الاختيار (σ) والإسقاط (π) في الجبر العلائقي.

5- الخصائص المساعدة

(IDENTITY) : لإنشاء أعمدة بترقيم تلقائي.
 (AS) : لإعطاء أسماء مستعارة للحقول.
 (' ') : (علامات تنصيص): للتعامل مع النصوص.
 (*) : لاختيار جميع الأعمدة

6- عمليات على البيانات

العمليات الحسابية : + - *
 الدوال النصية: LOWER, UPPER, LENGTH
 المقارنات المتقدمة LIKE, BETWEEN, IS NULL

7- استعلامات متعددة الجداول

UNION, INTERSECT, EXCEPT (+ ALL)
 و هي تعادل عمليات الاتحاد U ، التقاطع $\cap$ ، والفرق - في الجبر العلائقي.

أولاً أنواع البيانات

فائدة أريسيزية !

أنواع البيانات هي التي تحدد نوع القيم التي يمكن تخزينها داخل أعمدة الجدول،
 وتساعد في ضمان دقة البيانات وكفاءتها.

1. الأنواع النصية (Character/String Data Types)

النوع	الوصف	مثال
CHAR(n)	سلسلة محارف بطول ثابت n . يُملأ الفراغ إذا كانت القيمة أقصر.	CHAR(5) = 'AB'
VARCHAR(n)	سلسلة محارف بطول متغير يصل إلى n. أكثر كفاءة من CHAR	VARCHAR(10) = 'Hello'
TEXT	سلسلة نصية طويلة (قديمة - غير مفضلة في الإصدارات الحديثة).	فقرة نصية طويلة

ملاحظة: يفضل استخدام VARCHAR(MAX) بدلاً من TEXT .

2. الأنواع العددية (Numeric Data Types)

النوع	الوصف	مثال
INT	عدد صحيح (من -2,147,483,648 إلى 2,147,483,647)	5000
SMALLINT	عدد صحيح صغير	32000
BIGINT	عدد صحيح كبير جداً	90000000000
DECIMAL(p,s)	عدد عشري بدقة ثابتة	DECIMAL(5,2) = 123.45
FLOAT	عدد عشري بدقة عائمة (Floating Point)	3.14159
BIT	قيمة منطقية (0 أو 1)	0 أو 1

3. أنواع التاريخ والوقت (Date and Time Types)

النوع	الوصف	مثال
DATE	تاريخ فقط (YYYY-MM-DD)	2025-06-18
TIME	وقت فقط (HH:MM:SS)	14:30:00
DATETIME	تاريخ ووقت	2025-06-18 14:30
SMALLDATETIME	مثل DATETIME ولكن بدقة أقل	2025-06-18 14:30

4. أنواع أخرى شائعة

النوع	الوصف
BOOLEAN	غير مدعوم بشكل مباشر في SQL Server ، لكن يستخدم BIT بدلاً منه
UNIQUEIDENTIFIER	لتخزين معرفات فريدة (GUIDs)
BINARY / VARBINARY	لتخزين البيانات الثنائية مثل الصور أو الملفات

كيف نختار نوع البيانات المناسب؟

اختر النوع :

- ✓ الذي يحفظ الدقة المناسبة للبيانات مثل DECIMAL للأموال.
- ✓ والذي يحفظ المساحة مثلاً SMALLINT بدلاً من INT إن أمكن.
- ✓ الذي يتوافق مع نوع البيانات المتوقع مثلاً DATE وليس VARCHAR لتواريخ.

ثانياً إدارة هيكل الجداول

تتم كل عملية من عمليات إدارة هيكل الجداول (Create, Drop, Alter) في SQL Server بطريقتين :

- (1) بالكود (SQL Script)
- (2) عن طريق واجهة (SQL Server Management Studio (SSMS وهي شائعة أكثر في الحياة العملية .

1. أولاً: إنشاء جدول (CREATE TABLE):

الطريقة بالكود (SQL):

```
CREATE TABLE Employees (
    emp_id INT PRIMARY KEY,
    full_name VARCHAR(100),
    hire_date DATE,
    salary DECIMAL(10, 2),
    is_active BIT
);
```

الطريقة من واجهة SSMS :

1. افتح SQL Server Management Studio (SSMS).
2. من قاعدة البيانات التي تريد العمل عليها، اضغط زر اليمين على "Tables".
3. اختر "New Table..."
4. أضف الأعمدة واحدة واحدة:

- الاسم (Column Name)
- النوع (Data Type)
- هل يقبل قيمة فارغة؟ (Allow Nulls)

5. لحفظ الجدول:

- اضغط Ctrl + S
- أدخل اسم الجدول مثل (Employees)
- اضغط OK

2. ثانيًا: تعديل جدول (ALTER TABLE)

الطريقة بالكود (SQL):

إضافة عمود جديد:

```
ALTER TABLE Employees
ADD department VARCHAR(50);
```

تعديل نوع عمود:

```
ALTER TABLE Employees
ALTER COLUMN salary FLOAT;
```

حذف عمود:

```
ALTER TABLE Employees
DROP COLUMN department;
```

الطريقة من واجهة SSMS :

1. في SSMS ، افتح قائمة "Tables" ، ثم اضغط باليمين على الجدول المطلوب.
2. اختر "Design".
3. يمكنك:

- إضافة أو تعديل الأعمدة.
- حذف عمود بالضغط عليه ثم زر Delete .
- 4. عند الانتهاء، اضغط Ctrl + S للحفظ.

3. ثالثًا: حذف جدول (DROP TABLE)

الطريقة بالكود (SQL):

```
DROP TABLE Employees;
```

الطريقة من واجهة SSMS :

1. في SSMS ، من قائمة الجداول، اضغط زر اليمين على الجدول المراد حذفه.
2. اختر Delete.
3. ستظهر نافذة تأكيد — اضغط OK لإتمام الحذف.



Note !

ما هي القيود (Constraints) ؟

القيود هي قواعد تُفرض على الأعمدة في الجداول، بهدف:

- ضمان صحة البيانات.
- منع إدخال قيم غير منطقية أو غير مسموح بها.
- فرض العلاقات بين الجداول (عبر المفاتيح).

الأنواع الأساسية للقيود :

نوع القيد	الفرض
PRIMARY KEY	يضمن أن القيم في العمود فريدة وغير فارغة.
FOREIGN KEY	يربط جدولًا بجدول آخر.
UNIQUE	يضمن أن القيم فريدة (لكن يمكن أن تكون فارغة).
NOT NULL	يمنع ترك العمود فارغًا.
CHECK	يفرض شرطًا منطقيًا على القيمة.
DEFAULT	يضع قيمة افتراضية إذا لم يتم إدخال قيمة.

القيود (Constraints)

إضافة القيود باستخدام (SQL):

أثناء إنشاء الجدول:

```
CREATE TABLE Students (
  student_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  age INT CHECK (age >= 18),
  city VARCHAR(50) DEFAULT 'Damascus'
);
```



أو بعد إنشاء الجدول :

-- إضافة مفتاح أساسي

```
ALTER TABLE Students
ADD CONSTRAINT PK_Students PRIMARY KEY (student_id);
```

-- إضافة مفتاح خارجي

```
ALTER TABLE Enrollments
ADD CONSTRAINT FK_Enrollments_Students
FOREIGN KEY (student_id) REFERENCES Students(student_id);
```

-- إضافة شرط تحقق (Check)

```
ALTER TABLE Students
ADD CONSTRAINT CHK_Students_Age CHECK (age >= 18);
```

-- إضافة قيمة افتراضية

```
ALTER TABLE Students
ADD CONSTRAINT DF_Students_City DEFAULT 'Damascus' FOR city;
```

تعليمية تعديل

```
ALTER TABLE Students DROP CONSTRAINT CHK_Students_Age;
```

يمكن حذف قيد باستخدام:

الطريقة من واجهة SSMS :

1. افتح SSMS
2. اضغط بالزر اليمين على الجدول المطلوب، واختر Design
3. قم بما يلي:

- لتحديد مفتاح أساسي: اضغط باليمين على العمود واختر Set Primary Key.
- لتحديد NOT NULL: أزل علامة "Allow Nulls" من العمود.
- لتحديد Default: أدخل القيمة في خانة Default Value or Binding

4. لإضافة مفتاح خارجي أو Check :

- من الشريط العلوي اختر Relationships أو Check Constraints.
- أضف القيد وحدد العمود والجدول المرتبط إن وُجد.

5. احفظ التعديلات (Ctrl + S) .

ثالثاً : تعديل البيانات (Data Manipulation)

DELETE

لحذف بيانات

UPDATE

لتعديل بيانات موجودة.

INSERT

لإضافة بيانات جديدة

1. أولاً: إضافة البيانات INSERT INTO

الصيغة العامة:`INSERT INTO table_name (column1, column2, ...)``VALUES (value1, value2, ...);`

مثال:

```
INSERT INTO Students (student_id, name, age, city)
VALUES (1, 'Ali Ahmad', 20, 'Damascus');
```

```
INSERT INTO Students
VALUES (2, 'Sara Youssef', 22, 'Homs');
```

في حال إدخال قيم جميع الأعمدة
و بنفس الترتيب الموجود في الجدول
يمكنك حذف أسماء الأعمدة:

2. ثانياً: تعديل البيانات UPDATE

الصيغة العامة:`UPDATE table_name``SET column1 = value1, column2 = value2, ...``WHERE condition;`

مثال:

```
UPDATE Students
SET city = 'Aleppo'
WHERE student_id = 1;
```

ملاحظة هامة: إذا نسيت جملة WHERE

سيتم تعديل قيم جميع الصفوف و جعل مدينة كل الطلاب Aleppo !!!

3. ثالثاً: حذف البيانات DELETE

الصيغة العامة:

```
DELETE FROM table_name
WHERE condition;
```

```
DELETE FROM Students
WHERE student_id = 2;
```

مثال:

انتبه بدون WHERE، ستحذف جميع الصفوف.

ملاحظات مهمة :

- تأكد دائماً من وجود شرط WHERE عند استخدام DELETE أو UPDATE
- يمكنك استخدام SELECT قبل تنفيذ الأمر للتأكد (نقوم باختيار البيانات التي نريد حذفها للتأكد أنه سيتم حذف الشيء الذي نهدف الى حذفه)
- ستحدث عن التعليمة SELECT في الفقرة القادمة.

رابعاً : أوامر الاستعلام الأساسية

وهي مجموعة الأوامر التي نستخدمها لاسترجاع البيانات من الجداول، و تعتبر حجر الأساس في التعامل مع قواعد البيانات.
SELECT →FROM →WHERE

الصيغة الأساسية :

```
SELECT column1, column2, ...
FROM table_name
WHERE condition
```

مثال :

```
SELECT name, age
FROM Students
WHERE city = 'Damascus';
```

- لتحديد الأعمدة التي تريد عرضها.
- لتحديد الجدول الذي ستأخذ منه البيانات.
- لتحديد شرط يُفلتر البيانات ويعرض فقط ما يطابقه.

انتبه من الخلط بين SELECT في الجبر العلاقتي و SELECT في sql server

خامساً: الخصائص المساعدة و العمليات على البيانات

هذه الخصائص لا تعتبر أوامر رئيسية، لكنها تستخدم داخل الاستعلامات لجعل النتائج أكثر دقة وتنظيماً، أو لتعديل طريقة عرض البيانات .

1. DISTINCT – حذف التكرارات

تستخدم لجلب قيم غير مكررة في العمود (أو الأعمدة) المحددة.

مثال:

```
SELECT DISTINCT city FROM Students;
```

يعرض جميع المدن التي يوجد طلاب منها مرة واحدة فقط.

2. * – اختيار جميع الأعمدة

تستخدم عندما تريد جلب كل الأعمدة من الجدول.

مثال:

```
SELECT * FROM Students;
```

تعرض كل الأعمدة: student_id, name, age, city ..

3. ' ' – القيم النصية (Strings)

توضع القيم النصية دائماً داخل علامات اقتباس مفردة ' ' (single quote)

مثال:

```
SELECT * FROM Students WHERE city = 'Damascus';
```

انتبه ! لا تكتب Damascus بدون علامات الاقتباس أو باستخدام علامات مزدوجة (" ").

4. AS – الاسم المستعار (Alias)

تستخدم لإعطاء اسم مؤقت (مستعار) للعمود أو الجدول في النتيجة المعروضة. (rename في الجبر العلاقتي)

مثال:

```
SELECT name AS student_name, age + 1 AS age_next_year  
FROM Students;
```

يسهل قراءة النتيجة، خاصة عند استخدام حسابات أو دوال.

5. العمليات الحسابية: +, -, *, /

يمكن إجراء عمليات رياضية مباشرة داخل الاستعلام.

مثال:

```
SELECT full_name, salary * 12 AS yearly_salary
FROM Employees;
```

يحسب الراتب السنوي لكل موظف.

6. LIKE – التطابق الجزئي (نمط)

تستخدم للبحث داخل النصوص باستخدام نمط (Pattern).

- % تعني أي عدد من الأحرف (بما فيهم لا شيء)
- _ تعني حرف واحد فقط

أمثلة :

```
SELECT * FROM Students WHERE name LIKE 'A%';
```

أسماء الطلاب التي تبدأ ب A

```
SELECT * FROM Students WHERE name LIKE '%A%';
```

أسماء الطلاب التي تحوي A

```
SELECT * FROM Students WHERE name LIKE '%A_';
```

أسماء الطلاب التي فيها الحرف ما قبل الأخير A

```
SELECT * FROM Students WHERE name LIKE '%A__';
```

أسماء الطلاب التي حرف ال A فيها يقع قبل آخر حرفين

7. LOWER() , UPPER() , LEN() LENGTH() , + , ||

الربط بين الأعمدة (|| أو + في SQL Server):

```
SELECT first_name + ' ' + last_name AS full_name FROM Employees;
```

في SQL Server، نستخدم + للربط النصي.

وليس || كما هو شائع في بعض الأنظمة الأخرى (مثل PostgreSQL أو Oracle).

```
SELECT UPPER(name), LOWER(city) FROM Students;
```

تحويل النصوص :

KHALED – homs/ALI - aleppo...

```
SELECT LEN(name) FROM Students;
```

طول النص:

8. BETWEEN – البحث ضمن مجال (Range)

يستخدم BETWEEN للبحث عن قيمة ضمن مجال معين، وغالبًا ما تستخدم مع القيم الرقمية أو التواريخ.

الصيغة الأساسية :

column_name BETWEEN value1 AND value2

و تشمل القيمتين الطرفيتين (value1 و value2). (مجال مغلق)

أمثلة :

بين قيم رقمية.

```
SELECT * FROM Students
WHERE age BETWEEN 18 AND 25;
```

بين تواريخ.

```
SELECT * FROM Orders
WHERE order_date BETWEEN '2023-01-01' AND '2023-12-31';
```

يمكن استخدام المعاملات المنطقية < و > بدلاً من BETWEEN، لكنها تتطلب كتابة أطول.

استعلام العمر السابق باستخدام BETWEEN يعادل تمامًا :

```
SELECT * FROM Students
WHERE age >= 18 AND age <= 25;
```

9. IS NULL – التحقق من القيم الفارغة

في SQL ، القيمة الفارغة (NULL) لا تعني صفرًا أو فراغًا، بل تعني "لا يوجد قيمة".
ولا يمكن التحقق منها باستخدام = أو < أو >، بل باستخدام IS NULL أو IS NOT NULL.

الصيغ :

column_name IS NULL

column_name IS NOT NULL

مثال:

الطلاب الذين مدينتهم غير مسجلة (لا يوجد لها قيمة)

```
SELECT * FROM Students
WHERE city IS NULL;
```



10. IDENTITY – ترقيم تلقائي للمصفوف

IDENTITY تستخدم لجعل SQL Server يقوم تلقائياً بترقيم الأعمدة (غالباً المفتاح الأساسي primary key) عند إدخال صف جديد.

الصيغة داخل إنشاء جدول:

CREATE TABLE Students (

student_id INT IDENTITY(1,1) PRIMARY KEY,

name VARCHAR(100),

age INT);

IDENTITY(1, 1)

أي أن الصف الأول يأخذ 1 student_id = 1
والثاني 2، وهكذا...

بدء الترقيم من 1

يزيد ب 1 كل مرة

خصائص IDENTITY :

- لا تدخل قيمته يدوياً عند الإضافة (SQL Server) يقوم بذلك تلقائياً
- يستخدم غالباً للمفاتيح الأساسية.
- يمكنك تغيير قيمة البداية أو الزيادة، مثل IDENTITY(100,5)



تهجير (ترحيل) قواعد البيانات Database migration :

تهجير هو عملية نقل البيانات من قاعدة بيانات إلى أخرى (سواء على نفس الخادم أو إلى خادم جديد)، مع الحفاظ على نفس البنية أو تطويرها.

في هذه الحالة:

إذا كانت البيانات المنقولة تحتوي بالفعل على قيم لمعرف الصف (ID)، نحتاج إلى ضبط IDENTITY لتبدأ بعد آخر رقم موجود، أو بمقدار مختلف لتجنب التعارض مع البيانات السابقة.

مثال عملي:

إذا كان آخر student_id في النظام القديم هو 99، نبدأ الترقيم في النظام الجديد هكذا:

IDENTITY(100,1)

سادساً : الاستعلامات متعددة الجداول

UNION, INTERSECT, EXCEPT [+ ALL]

تُستخدم للجمع بين نتائج استعلامات متعددة بشرط أن تكون نفس عدد الأعمدة
و نفس النوع و ترتيب الأعمدة متطابق.

1. UNION – دمج النتائج بدون تكرار

يجمع بين نتائج استعلامين ويُزيل التكرارات تلقائياً.

الصيغة :

```
SELECT column1 FROM TableA
```

```
UNION
```

```
SELECT column1 FROM TableB;
```

مثال :

```
SELECT city FROM Students
UNION
SELECT city FROM Employees;
```

يعرض قائمة بكل المدن التي يظهر فيها طلاب
أو موظفون بدون تكرار.

2. UNION ALL – دمج النتائج مع التكرار

نفس UNION لكن لا يحذف القيم المكررة.

3. INTERSECT – تقاطع النتائج (القيم المشتركة)

يعرض فقط الصفوف التي تظهر في الاستعلامين معاً.

مثال :

```
SELECT city FROM Students
INTERSECT
SELECT city FROM Employees;
```

يُظهر فقط المدن التي يوجد فيها طلاب و موظفون معاً.

4. EXCEPT – الفرق (ما في الأول فقط)

يعرض الصفوف التي تظهر في الاستعلام الأول فقط ولا تظهر في الثاني.

مثال :

```
SELECT city FROM Students
EXCEPT
SELECT city FROM Employees;
```

يعرض المدن التي يوجد فيها طلاب فقط وليس فيها موظفون.

ALL مع هذه العمليات:

▪ UNION ALL :

لعرض كل الصفوف بما فيها المكررة.

▪ INTERSECT ALL و EXCEPT ALL :

غير مدعومتين في SQL Server مباشرة، لكنها مفاهيم موجودة في بعض الأنظمة الأخرى.

أفكار لاستخدام SELECT مع قيم ثابتة أو ربط نصوص :

(1) 'SELECT 'x' - طباعة قيمة ثابتة

يمكنك استخدام SELECT لطباعة أي قيمة ثابتة دون الحاجة إلى جدول.

مثال :

```
SELECT 'Hello, SQL!';
```

النتيجة :

(No column name)
Hello, SQL!

ملاحظة : من الأفضل دائماً استخدام AS لتسمية العمود:

```
SELECT 'Hello, SQL!' AS greeting_sql ;
```

greeting_sql
Hello, SQL!

(2) SELECT 5 + 10 - العمليات الحسابية بدون جداول

يمكن تجربة الجمع، الطرح، الضرب، القسمة مباشرة داخل SELECT .

```
SELECT 5 + 10 AS result;
```

result
15

3) SELECT 'Hello ' + name - ربط النصوص

يمكن دمج نص ثابت مع قيم الأعمدة، مثل عرض جملة ترحيبية لكل مستخدم.

```
SELECT 'Hello ' + name AS greeting
FROM Students;
```

greeting
Hello Ali
Hello Sara

هذه الطريقة تستخدم كثيرًا في الرسائل التلقائية.

4) SELECT name + ' (' + city + ') ' - دمج نصوص متعددة

```
SELECT name + ' (' + city + ') ' AS full_info
FROM Students;
```

full_info
Ali (Damascus)
Sara (Homs)

هذه الصيغة ممتازة عند عرض بيانات مختصرة في التطبيقات.

إضافة مفتاح خارجي (Foreign key) الى جدول

المفتاح الخارجي (Foreign key) هو قيد (Constraint) يُستخدم لربط عمود أو مجموعة أعمدة في جدول بالمفتاح الأساسي (Primary key) في جدول آخر. الغرض منه: ضمان أن القيمة المُدخلة في هذا العمود **موجودة مسبقًا** في الجدول المرتبط.

الصيغة :

```
FOREIGN KEY (foreign key column name)
REFERENCES Table_Name(Primary_key_of_this_table)
==
FOREIGN KEY (اسم_العمود_في_الجدول_الحالي)
REFERENCES (المفتاح_الأساسي_في_ذلك_الجدول) اسم_الجدول_المرتبط
```

مثال :

```
FOREIGN KEY (student_id)
REFERENCES Students(student_id)
```

ملاحظة وتحذير شائع :

عند استخدام الصيغة السابقة يجب الانتباه الى أمرين :

1. وجود العمود داخل الجدول الحالي:

العمود student_id يجب أن يكون معرفاً مسبقاً في الجدول الذي تنشئ فيه المفتاح الخارجي و ليكن (grades) إذا لم يكن موجوداً، عليك إضافته أولاً:

```
ALTER TABLE Grades
ADD student_id INT;
```

ثم تضيف القيد باستخدام ALTER TABLE + ADD CONSTRAINT.

2. وجود الجدول والعمود المشار إليه في REFERENCES:

يجب أن يكون الجدول Students موجوداً فعلياً، ويجب أن يكون العمود student_id فيه عبارة عن مفتاح أساسي (PRIMARY KEY) أو مفتاح فريد (UNIQUE)..
- إذا لم تتحقق هذه الشروط، فسيعطي SQL Server خطأ عند تنفيذ الأمر.

كيف تتعامل قواعد البيانات مع الفراغات (Gaps) في البيانات

مع مرور الوقت، عند استخدام قاعدة البيانات في التطبيقات الحقيقية (مثل أنظمة الطلاب أو الطلبات أو الحسابات)، تحدث السيناريوهات التالية:

- يتم إدخال صفوف جديدة.
- ثم يتم حذف بعض الصفوف أو تعديلها.

هذا يؤدي إلى وجود فراغات (Gaps) في ترقيم السجلات، خاصة في الأعمدة التي تعتمد على IDENTITY أو INT كأرقام تسلسلية.

المشكلة العملية:

إذا كان العمود مثل order_id INT IDENTITY(1,1) يُستخدم للترقيم، ومع مرور الوقت تم حذف آلاف السجلات فإن الأرقام (IDs) المفقودة لا يُعاد استخدامها تلقائياً.

ومع زيادة البيانات، قد تقترب من الحد الأقصى لنوع البيانات مثلاً (INT : 2,147,483,647)، رغم وجود فراغات كبيرة غير مستخدمة .

هنا يأتي دور ال Database Admin (DBA) :

المسؤول عن قاعدة البيانات يمكنه:

- ✓ إعادة هيكلة قاعدة البيانات.
- ✓ أو إعادة ضبط الترقيم (reseeding) لتقليل الهدر.
- ✓ أو تحويل نوع العمود من INT إلى BIGINT إذا كان النمو كبيراً.
- ✓ أو استخدام أدوات الصيانة لفحص الفجوات وتنظيم الاستخدام الداخلي للصفوف والمساحة.

الخلاصة : في الحياة العملية، إدارة البيانات لا تقتصر فقط على الإدخال والاستعلام، بل تشمل أيضاً مراقبة الأداء، تنظيم التخزين، والتأكد من الاستخدام الأمثل للموارد

ميزة Generate Script :

بعد تنفيذ العملية عبر الواجهة، يمكننا ببساطة الضغط على:

Script → Script Action to → New Query Window

ليقوم SSMS تلقائياً بتوليد كود SQL كامل يقابل ما قمنا به بالواجهة!

نهايةً في هذه المحاضرة تعلمنا كيفية كتابة أوامر SQL يدوياً، لكن في الواقع العملي داخل الشركات، أغلب هذه العمليات يتم تنفيذها غالباً عبر واجهات رسومية من خلال برنامج [SQL Server Management Studio \(SSMS\)](#) (ذكرنا كيفية إجراء بعض العمليات باستخدامه في بداية هذه المحاضرة).



انتهت المحاضرة الخامسة

Go to part 2